

Contributing to firm-bot

Mine-FNL

Contributing to firm-bot

Thanks for considering a contribution. firm-bot is meant to be the easiest way to ship a citation-grounded chatbot over a firm's private documents. Issues and PRs that move that goal forward are welcome.

Ground rules

1. **Data isolation is sacred.** Never change the per-firm data layout in a way that could leak chunks between firms. The chroma collection name, the BM25 pickle, and the filesystem root must remain per-FirmConfig.slug.
2. **Citations are mandatory.** Any PR that relaxes the "every claim cites a source" invariant will be closed. If the model can't ground a claim, it must say so plainly.
3. **Local-first by default.** Anything that ships data off-machine needs a clear, opt-in flag.
4. **No bare `except:` clauses.** Use specific exception types. If a broad `except Exception` is genuinely required, log the error before re-raising or returning a fallback.
5. **No silent failures.** If an extractor returns 0 documents from a file, the operator must hear about it (a log line at minimum).

Development setup

```
git clone https://github.com/<you>/firm-bot
cd firm-bot
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

The dev extra installs `pytest`, `pytest-cov`, `ruff`, `mypy`.

Tooling

- **Lint / format:** `ruff check firm_bot tests eval` and `ruff format --check firm_bot tests eval`
- **Type-check:** `mypy firm_bot`
- **Tests:** `pytest tests -q`
- **Tests with coverage:** `pytest tests --cov=firm_bot --cov-report=term-missing`

CI runs the same four commands on macOS + ubuntu. A PR that fails any of them is not ready for review.

Code style

- 4-space indent, line length 100.
- Type hints on all public functions; use `from __future__ import annotations` at the top of every module.
- One blank line between top-level definitions; two between top-level sections.
- Docstrings on every module and every public function. Use imperative mood for one-line summaries ("Run the X" not "Runs the X").
- Pydantic for all API request/response models. Raw dicts only inside `_internal` helpers.
- New CLI flags go under `firm_bot/cli.py`. New env vars go under `firm_bot/config.py` with a clear default and a comment explaining when to override.

Adding a new extractor

The extractor protocol is implicit:

```
def extract_<format>(path: Path) -> list[Document]:  
    """..."""
```

To add a new format (e.g. `.html`):

1. Implement `firm_bot/ingest/<format>.py` returning `list[Document]`.
2. Add the suffix to `SUPPORTED_SUFFIXES` in `firm_bot/ingest/common.py` and add a branch in `dispatch()`.
3. Add a test in `tests/test_ingest_<format>.py` covering at least: happy path, malformed file, empty file.
4. Update the README's "Supported formats" list.

Adding a new retrieval signal

Hybrid retrieval uses BM25 + dense + RRF. If you want to add a third signal (e.g. cross-encoder reranker, knowledge-graph expansion):

1. Implement the new search under `firm_bot/retrieve/<signal>.py` returning a list of `(chunk_id, score, rank)` like the existing `bm25_search` and `dense_search`.
2. Add a weight to `RootConfig (<signal>_weight)`.
3. Wire it into `hybrid_search()` with its own RRF contribution.
4. Add a test that compares the new signal vs the existing two on a small fixture.

Pull request checklist

- ☐ `ruff` check is clean

- ☐ mypy firm_bot is clean
- ☐ pytest tests passes
- ☐ Coverage has not dropped (see CI report)
- ☐ Updated README / docs if the user-facing surface changed
- ☐ Added at least one test for the new behaviour
- ☐ Commits describe what is IN them, not what comes next

Reporting a security issue

See [SECURITY.md](#). Do not file a public issue for vulnerabilities that affect firm data confidentiality.